Handout for Laboratory Session 5

1. Instructions :

Generation of scanner and parser	Test the generated Parser

$ lex lex-script
$ yacc -dv yacc-script
$ gcc lex.yy.c y.tab.c -o parser
	$ ./ parser < input.txt > output.txt
	where "input.txt" is file containing input and the results produced by the parser is saved in the file, "output.txt"

2. How to manually construct a parse tree using the grammar actions?

Consider problem P2 of Lab 4, which deals with declarations of C, and the 2 scripts as shown below.

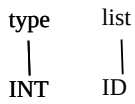
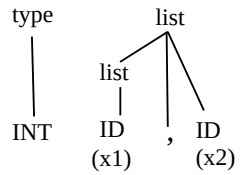
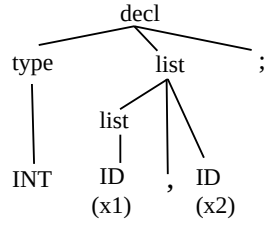
Lex script "lab4-p2.l"

```
ws [ \t\n]+
letter [A-Za-z_]
digit  [0-9]
id {letter}({letter}|{digit})*
%%
{ws}   {;/* No action */}
"int"    {return INT;}
"float" {return REAL;}
"char"  {return CHAR;}
{id}      {return ID;}
";"         {return ';';}
","         {return ',';}
.        {printf(" Unrecognized character %s\n",
          yytext);}
```

Yacc script "lab4-p2.y"

```
%token INT ID REAL CHAR
/* Grammar follows */
%%
dlist  : dlist decl {printf("Rule used : dlist -> dlist decl\n");}
       | decl {printf("Rule  used : dlist -> decl\n");}
       ;
decl     : type list ';'
 {dcount++;
   printf("Rule used : decl → type list ; Declaration %d \n",
           dcount);}
       ;
type:    INT          {printf("Rule used : type -> INT\n");}
       | REAL       {printf("Rule used : type -> REAL\n");}
       | CHAR       {printf("Rule used : type -> CHAR \n");}
       ;
list :   list ',' ID   {printf("Rule used: list -> list , ID \n");}
       | ID          {printf("Rule used : list -> ID \n");}
       ;
%%
```

Examine the following table for step by step details.

Input	Token(s) from scanner	Output from Parser	Parse Tree using the output
int x1, x2; char ch1, _ch2;	int	Rule : type → INT	
	ID for x1	Rule : list → ID	
	, ID (for x2)	Rule : list → list , ID	
	;	Rule : decl-> type list ;